Refactoring the Operations Kernel: Transitioning from Monolithic Governance to a Distributed Index-Leaf Architecture via the Elevation of the Payload Tier

1. Executive Summary: The Imperative for Structural Evolution

The "Stela" repository, a sophisticated repo-resident artificial intelligence environment, stands at a critical inflection point. As documented in the forensic analysis of the "Refactoring Ops Kernel For New Architecture" [1], the system is currently constrained by an archaic "Filing Doctrine" that was optimized for human-centric readability rather than high-throughput agentic operations. The central impediment identified is the semantic and structural misclassification of the storage/ directory as "Trash"—a disposable holding area—rather than as a foundational "Payload Tier" essential for state persistence, context management, and operational continuity.[1]

This report presents a comprehensive research analysis and implementation roadmap for instantiating a new architectural paradigm: the **Distributed Index-Leaf Filing Doctrine**. This paradigm shift is necessitated by the observation that monolithic ledgers, such as the Policy of Truth (PoT.md) and State of Play (SoP.md), are succumbing to "Ledger Bloat" and "Context Exhaustion".[1] As these governance surfaces grow unbounded, they degrade the signal-to-noise ratio required for Large Language Models (LLMs) to function effectively, increasing both computational cost and the probability of hallucinatory drift.

The proposed architecture elevates storage/ from a second-class receptacle to a first-class state layer, subdivided into specialized domains compliant with the Filesystem Hierarchy Standard (FHS): archives/ (Cold Store), tmp/ (Resume Store), and handoff/ (Hot Interconnect).[1] By rigorously migrating ephemeral artifacts into these governed sub-systems and excising all references to "Trash" from the constitutional documents [1], the repository will transition from a stateless execution environment to a state-aware persistence engine. This evolution is supported by advanced Git mechanisms—specifically sparse-checkout for factory isolation [3] and skip-worktree for local configuration persistence [4]—creating a deterministic substrate capable of supporting complex, multi-session agentic workflows.

2. Forensic Architecture Audit: The Entropy of the Status Quo

2.1 The Limits of the Monolithic Ledger Model

The current governance architecture of Stela is predicated on a "Monolithic Ledger" model. The Policy of Truth (PoT.md) serves as the immutable constitution, while the State of Play (SoP.md) and Proof of Work (PoW.md) act as linear, append-only history logs.[1] In the nascent stages of the project, this model provided high referential integrity; every significant action was recorded sequentially in a single file, offering a clear, auditable timeline.

However, forensic analysis of the repository dump [1] reveals that this linearity has become a liability. The PoW.md file, designed to store execution proofs, lacks a native rotation mechanism. As agents execute Dispatch Packets (DPs), they append verbose receipts—hashes, diff stats, and verification outputs—directly to the active index. This results in "Ledger Bloat" [1], where the file size grows linearly with operational tempo.

For an LLM-based Contractor, this presents a "Context Exhaustion" challenge. To understand the current state of the repository, the agent must ingest the entire history. As the ledger exceeds the effective context window of the model (or moves into the "lossy middle" of the window), the agent loses the ability to discern valid precedents from obsolete history. The "Refactoring Ops Kernel" report explicitly identifies this friction, noting that the monolithic approach increases the "cost for Large Language Models (LLMs) to ingest governance context".[1] The system is effectively drowning in its own history.

2.2 The "Trash" Fallacy: Semantic Misalignment of Storage

The most critical architectural flaw identified is the semantic labeling of the storage/ directory. The current PoT.md explicitly defines the Filing Doctrine as follows:

- ops/ = Run
- docs/ = Explain
- projects/ = Work
- opt/ = Isolate
- **storage/ = Trash (local artifacts, never canon)**.[1]

This definition constitutes a "semantic misalignment" that effectively cripples the system's ability to manage state.[1] By labeling storage/ as "Trash," the constitution implies that the data within it is disposable, ephemeral, and devoid of value. However, a structural analysis of the dump-full-main-80bb3afd.txt [1] reveals that storage/ actually contains high-value artifacts:

1. **Context Snapshots:** The storage/dumps/ directory holds serialized states of the repository used for rehydration and error recovery.[1]
2. **Operational Receipts:** The storage/handoff/ directory contains

DP-OPS-XXXX-RESULTS.md files, which are the primary evidence of completed work.[1]
3. **Governance Whitelists:** The file storage/dp/active/allowlist.txt controls the "hard gate" for file modifications, a critical security control.[1]

Treating these artifacts as "Trash" prevents the development of sophisticated retention, indexing, or protection policies. If the system views storage/ as a waste bin, it cannot justify the implementation of a "Resume Store"—a safe harbor for agent memory between sessions. This forces agents to be stateless, obliterating their "thought process" (intermediate reasoning) at the end of every session, and requiring expensive re-computation for complex tasks.

2.3 Functional Overloading and Brittle Enforcement

The forensic audit further uncovers "Functional Overloading" within the Ops Kernel binaries. The ops/bin/project script conflates the instantiation of new projects with governance verification, creating tight coupling that makes refactoring difficult.[1] Similarly, ops/bin/template mixes the expansion of variable definitions with the synthesis of manifests, violating the Single Responsibility Principle.

On the enforcement side, the CI/CD workflows (.github/workflows/) rely on "Brittle Enforcement" mechanisms.[1] For instance, pow_policing.yml likely uses regex patterns to check for ledger updates.[1] If the structure of the ledger changes slightly, or if the "Trash" definition is modified without updating these hardcoded paths, the policing logic fails. This brittleness is exacerbated by the lack of a centralized library for path definitions; paths like storage/dumps are likely hardcoded across multiple scripts (ops/bin/dump, ops/bin/prune, tools/verify.sh), making the migration of storage/ a high-risk operation.

Finally, the report notes the presence of "Dead Code" in tools/lint/, where unused scripts contribute to the "code-bearing surface".[1] This increases the friction of Proof of Work, as agents must verify files that are no longer operational.

3. Theoretical Framework: The Distributed Index-Leaf Paradigm

To resolve the entropy of the monolithic model, we propose a paradigm shift to a **Distributed Index-Leaf Architecture**. This concept borrows heavily from database theory (specifically B-Trees) and the Unix filesystem philosophy.

3.1 The Concept of Index and Leaf

In the current architecture, PoW.md acts as both the **Index** (the pointer to the latest state) and the **Leaf** (the storage of the historical data). The Distributed Index-Leaf paradigm decouples these roles:

- **The Hot Index:** A lightweight, active file (e.g., the truncated PoW.md) that contains only the most recent, relevant entries (e.g., the last 30 DPs). Its function is high-speed retrieval and current context establishment.
- **The Cold Leaf:** A static, immutable archive file (e.g., PoW-archive-2025-10.md) stored in storage/archives/. Its function is compliance, deep-dive forensics, and long-term history.

By separating the Index from the Leaf, we ensure that the "Hot Context"—the data loaded into the agent's immediate attention window—remains lean and high-signal.[1] The "Cold Context" is available on demand but does not burden the primary execution loop.

3.2 Storage as State (The Payload Tier)

The reclassification of storage/ from "Trash" to "Payload" is the physical manifestation of this theory. In this new model, **Storage is State**. It is the mutable memory of the repository.

- **Volatile State:** Data that exists only for the duration of a process (storage/handoff).
- **Persistent State:** Data that must survive a process restart (storage/tmp).
- **Archival State:** Data that is immutable and historical (storage/archives).

This categorization aligns with the **Filesystem Hierarchy Standard (FHS)**, which governs the layout of Unix systems.[2]

- storage/archives/ maps to /var/log or /var/lib (persistent data).
- storage/tmp/ maps to /var/tmp (temporary files preserved between reboots).
- storage/handoff/ maps to /var/run (runtime variable data).

By adopting FHS principles, we move from an ad-hoc directory structure to a standardized, universally understood hierarchy. This reduces the cognitive load on agents, as the directory names imply their function and retention policies.[2]

4. The New Filing Doctrine: PoT Physical Laws v2.0

The implementation of this architecture requires a rewriting of the "Physical Laws" in PoT.md. This section details the new definitions and the rationale behind each assignment.

4.1 Root Hierarchy Redefinition

The updated Filing Doctrine assigns strict functional roles to the top-level directories:

- **ops/ (Run):** The immutable kernel. Contains executable binaries, manifests, and automation logic. This is the "System Partition." It is read-only for standard agents and writable only during explicit "Kernel Refactoring" operations.[1]
- **docs/ (Explain):** The canon of truth. Contains manuals, specifications, and governance surfaces. This hierarchy is the "Knowledge Base." It explains *why* the system behaves as it does.
- **projects/ (Work):** The active payload. Contains user-facing code and applications. This

is the "User Space," distinct from the "Kernel Space" of ops/.

- **opt/ (Isolate):** The factory floor. Contains Just-In-Time (JIT) content, templates, and definition files (_factory). Crucially, this directory is excluded from the global context to prevent "Context Contamination," where agents confuse a template for a live instruction.[1]
- **storage/ (Payload):** The persistent state layer. Formerly "Trash," this is now the governed data plane. It is the only directory where agents are permitted to maintain state across sessions.[1]

4.2 The Storage Sub-Hierarchy Structure

The storage/ directory is further segmented to handle specific data lifecycles, replacing the flat "dump and run" approach.

4.2.1 storage/archives/ (The Cold Leaf Store)

This directory serves as the destination for the "Leaf" data rotated out of the monolithic ledgers.

- **Structure:** storage/archives/{root,logs,agents,tasks}.
- **Function:** When PoW.md exceeds its line limit, the ops/bin/prune utility moves the oldest entries here.
- **Retention:** Permanent. These files are never deleted, only compressed or moved to colder storage (off-repo) if necessary.
- **Research Insight:** Leveraging the GIT_ALTERNATE_OBJECT_DIRECTORIES concept [6], these archives could theoretically be moved to a separate "Archive Repo" to keep the main repository size manageable, though Phase 1 keeps them local for simplicity.

4.2.2 storage/tmp/ (The Resume Store)

This is the most critical addition for agentic workflows. It serves as the **Resume Container**.

- **Function:** Stores serialized agent memory ("thought vectors"), intermediate build artifacts, and dependency caches that must survive the "Freshness Gate" of a new session.[2]
- **Protection:** Unlike the system /tmp which may be cleared on reboot, storage/tmp/ is protected from standard pruning cycles. It is only cleared when a task is explicitly marked CLOSED or ABORTED.
- **Analogy:** It acts like /var/tmp in Linux systems, which preserves temporary files across system reboots.[2]

4.2.3 storage/handoff/ (The Hot Interconnect)

This directory acts as the IPC (Inter-Process Communication) socket between the Human Operator and the AI Contractor.

- **Function:** Stores OPEN prompts (instructions), RESULTS packets (evidence), and diff manifests.

- **Lifecycle:** Ephemeral. Once a task is closed and the evidence is logged in PoW.md (or archived), the handoff files are typically cleared to prepare for the next task.[1]

4.3 Axiomatic Shifts: Removing "Trash"

The removal of the word "Trash" is a psychological and technical necessity.

- **Psychological:** Calling it "Trash" implies that data loss is acceptable. This discourages agents from using it for complex, multi-step reasoning that requires storing intermediate state.
- **Technical:** "Trash" implies rm -rf. "Payload" implies mv and archive. The tooling logic must be refactored to default to preservation rather than destruction.[1]

5. Research Analysis: Mechanics of Persistence and Hygiene

To implement the new doctrine effectively, we must master the underlying Git mechanisms that control file tracking, ignoring, and isolation.

5.1 The .gitignore Precedence Strategy

A robust Filing Doctrine relies on a cascading .gitignore strategy to enforce boundaries without manual intervention. Git applies ignore patterns in a specific precedence order [7]:

1. Command line patterns.
2. .gitignore in the specific directory (and parents).
3. $GIT_DIR/info/exclude (local repo specific).
4. Global config.

For storage/, we require a **Directory-Specific Ignore Strategy**.[8] Reliance on the root .gitignore is insufficient because it creates a single point of failure and makes the storage/ directory definition opaque to sub-processes.

Recommendation: Place a .gitignore file inside storage/ and each of its subdirectories to define their specific physics.

The "Allow-List" Pattern for Archives:

We want to track the *structure* of the archives but not necessarily the *content* if it becomes too voluminous, or at least control it tightly. However, for Phase 1, we *do* want to track the archives (PoW-archive-*.md) because they are part of the audit trail.

Therefore, storage/archives/.gitignore should be configured to *allow* markdown files but ignore binaries.

The "Strict Ignore" Pattern for Resume (storage/tmp): The storage/tmp directory must be treated as "Local State." It should never be committed to the remote repository, as it may contain machine-specific paths, partial states, or potentially sensitive hallucinations (e.g., API keys generated during testing).[9] We employ the "Ignore All Except Self" pattern [10]:

Code snippet

```
# storage/tmp/.gitignore
# Ignore everything
*
# Except this gitignore file
!.gitignore
```

This ensures the directory exists in the repo structure (so agents can find it) but acts as a black hole for Git tracking.[11]

5.2 Mechanics of Isolation: Sparse Checkout for opt/

The opt/ directory contains factory definitions that are not needed for every task. In large repositories, loading these definitions into the working tree can slow down Git operations (git status, git checkout).[12] More importantly for agents, the presence of these files in the file tree can lead to "Context Contamination," where an agent reads a *template* for a task and mistakes it for an *active* task.[1]

Solution: Use git sparse-checkout to hide the opt/ directory by default.[3]

- **Cone Mode:** Using git sparse-checkout init --cone allows for efficient pattern matching.[13]
- **Workflow:**
  - **Standard Mode:** opt/ is hidden. Agent sees only ops/, docs/, projects/, storage/.
  - **Architect Mode:** When an agent is tasked with creating *new* agents or tasks, it executes git sparse-checkout add opt/ to reveal the factory floor.
    This physical isolation prevents hallucination by ensuring the agent simply *cannot see* the templates unless it has a specific mandate to use them.

5.3 Mechanics of Persistence: skip-worktree vs. assume-unchanged

A major challenge in the "Resume Store" is handling configuration files or state files that *must* exist in the repo (as templates) but are modified locally during execution. If an agent modifies a tracked config file in storage/tmp, Git will flag it as dirty.

Research compares two solutions:

1. **--assume-unchanged:** Tells Git "I promise I won't change this file." If the file *is* changed, Git may discard the changes during a reset or pull without warning.[14] This is unsafe for agent memory.
2. **--skip-worktree:** Tells Git "Pretend this file is up to date, even if it's different." It preserves local changes even during resets.[4]

Conclusion: For the Resume Store, if we ever need to track a "template" resume file but allow local modifications, **--skip-worktree** is the correct mechanism.[4] However, the safer path (as defined in 5.1) is to simply *ignore* the contents of storage/tmp entirely via .gitignore, avoiding the complexity of index manipulation. The skip-worktree method should be reserved for storage/dp/active/allowlist.txt if that file needs to be locally modified without committing (though typically, allow-lists *should* be committed).

6. Security and Telemetry Implications

Migrating "Trash" to "Storage" introduces new security vectors that must be managed via the Filing Doctrine.

6.1 The Telemetry Leak Vector

Agentic workflows often generate verbose logs and telemetry. In the Visual Studio Code environment, telemetry events are logged to local files.[16] If storage/ is just "Trash," an agent might dump these logs there. If the .gitignore is loose, these logs—potentially containing environment variables or snippet code—could be committed to the repo. **Mitigation:** The storage/.gitignore must explicitly exclude *.log, *.lock, and common telemetry patterns.[17]

6.2 The Secret Retention Risk

The "Resume Store" (storage/tmp) creates a risk of persistent secret exposure. If an agent hallucinates an API key into its resume file, that key persists on the disk.

Mitigation:

1. **Strict Ignore:** As mentioned, storage/tmp must be strictly ignored to prevent upstream commit.[9]
2. **Auto-Scrub:** The ops/bin/prune script needs a specific routine for storage/tmp. While it is a "Resume" store, it should have a Time-To-Live (TTL). If a resume file hasn't been touched in 7 days, prune should treat it as abandoned and scrub it to remove potential latent secrets.[1]

7. Implementation Roadmap

The transition to the Distributed Index-Leaf architecture is broken into three phases to

minimize disruption to active operations.

7.1 Phase 1: Structural Preparation (Non-Destructive)

This phase creates the physical hierarchy without changing the logic. It is purely additive.

- **Action:** Create storage/archives/root, storage/archives/logs, storage/archives/agents, storage/archives/tasks.
- **Action:** Create storage/tmp and storage/handoff.
- **Action:** Populate all new directories with .gitkeep files to enforce git persistence.[1]
- **Action:** Update tools/verify.sh to recognize these paths as valid "Payload" locations, effectively deprecating the "Trash" warnings.[1]
- **Action:** Implement ops/lib/scripts/common.sh to centralize path definitions (Registry Lookup), removing hardcoded paths from binaries.[1]

7.2 Phase 2: The Logic Switch (Migration)

This phase activates the new logic.

- **Action:** Refactor ops/bin/prune. Instead of deleting old entries, it must now:
  1. Count lines in PoW.md.
  2. If > 30 entries, extract the excess.
  3. Append excess to storage/archives/root/PoW-archive-YYYY-MM.md.
  4. Truncate PoW.md while preserving the header.[1]
- **Action:** Update ops/bin/dump. Add a --scope=state flag that *includes* storage/tmp and storage/archives for debugging purposes, but ensure the default --scope=platform explicitly *excludes* storage/archives to cure the Context Exhaustion problem.[1]

7.3 Phase 3: The Resume Container (Activation)

This phase enables stateful agents.

- **Action:** Update ops/lib/scripts/agent.sh to check for storage/tmp/resume.md on startup.
- **Action:** If present, inject the resume content into the context window.
- **Action:** Update PoT.md to officially recognize the "Resume" state as part of the agent's "Staffing Protocol".[1]

8. Detailed Tooling Refactor Specifications

8.1 Refactoring tools/verify.sh

The current verify.sh likely scans storage/ and warns on unexpected files.[1] This logic is now obsolete.

- **New Logic:** It must *mandate* the presence of storage/archives, storage/tmp, and storage/handoff.

- **Binary Checks:** It must rigorously enforce that no binary files exist in docs/ or ops/ (Filing Doctrine), forcing them into storage/dumps or storage/archives.[1]

8.2 Refactoring ops/bin/prune

The prune script transforms from a janitor to a librarian.

- **Guardrails:** It must check for uncommitted results before archiving. We cannot archive a ledger that implies a state that hasn't been committed yet. This "Results Guard" is critical for referential integrity.[1]
- **Targeting:** It needs flags: --target=pow (archives proof), --target=sop (archives history), --target=temp (scrubs resume store).[1]

8.3 Decomposing Overloaded Binaries

As noted in the audit [1], ops/bin/project and ops/bin/template are overloaded.

- **Proposal:** Split ops/bin/project into ops/bin/project-init (scaffolding) and ops/bin/project-audit (governance).
- **Proposal:** Split tools/verify.sh into tools/verify-tree.sh (fast structural check) and tools/verify-policy.sh (deep content inspection).[1] This allows "fast fail" checks in the inner loop of agent operation.

9. Conclusion

The "Refactoring Ops Kernel" initiative is not merely a cleanup; it is a fundamental maturation of the Stela repository's operating system. By moving from a "Monolithic Ledger" to a "Distributed Index-Leaf" architecture, we solve the Context Exhaustion problem that plagues long-running agent operations. By reclassifying "Trash" as "Payload," we unlock the capability for persistent agent memory ("Resume Store"), enabling a new class of complex, multi-session workflows.

This transition aligns Stela with industrial-strength standards like FHS and advanced Git workflows, transforming it from a simple file repository into a robust, state-aware agentic platform. The "Physical Laws" of PoT.md will no longer just describe where files *go*; they will define how the system *remembers*.

10. Appendix: Configuration Artifacts

10.1 Proposed storage/.gitignore Structure

Code snippet

```
# storage/.gitignore
# Deny by default: ignore everything
*
# Allow by exception: the structure itself
!.gitignore
!dp/
!archives/
!tmp/
!handoff/
!dumps/
```

10.2 Proposed storage/archives/.gitignore

Code snippet

```
# storage/archives/.gitignore
# Track the directory structure
!**/.gitkeep
# Track the archives themselves (they are cold context)
!*.md
# Ignore any compression artifacts if we decide to gzip later
*.gz
*.zip
```

10.3 Proposed storage/tmp/.gitignore (Resume Store)

Code snippet

```
# storage/tmp/.gitignore
# STRICT: Ignore everything. Local state only.
*
# Except the ignore file itself
!.gitignore
```

10.4 Proposed PoT.md Filing Doctrine Section

1.1 Filing Doctrine (Refactored)

The filesystem is the territory. Placement dictates function.

- ops/ = **Run**. Executable kernel. Binaries, manifests, automation. Immutable input.
- docs/ = **Explain**. Canon. Manuals, specifications, governance. Read-only reference.
- projects/ = **Work**. Active payload. User-facing code and applications.
- opt/ = **Isolate**. Factory floor. JIT templates/definitions excluded from global context.
- storage/ = **Payload**. Persistent state layer.
  - storage/archives/ = **Cold Store**. Immutable history leaves (Rotated Context).
  - storage/tmp/ = **Resume Store**. Session-persistent agent memory (Local State).
  - storage/handoff/ = **Interconnect**. Hot state exchange (Ephemeral).

Works cited

1. Refactoring Ops Kernel For New Architecture.md
2. Filesystem Hierarchy Standard - Wikipedia, accessed February 16, 2026, https://en.wikipedia.org/wiki/Filesystem_Hierarchy_Standard
3. git-sparse-checkout Documentation, accessed February 16, 2026, https://git-scm.com/docs/git-sparse-checkout
4. Don't git angry, skip in worktree | by Adi Ben David | Medium, accessed February 16, 2026, https://medium.com/@adi.ashour/dont-git-angry-skip-in-worktree-e9c77dec9d15
5. Filesystem Hierarchy Standard - Linux Foundation, accessed February 16, 2026, https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html
6. git Documentation - Git, accessed February 16, 2026, https://git-scm.com/docs/git
7. gitignore Documentation - Git, accessed February 16, 2026, https://git-scm.com/docs/gitignore
8. 2.2 Git Basics - Recording Changes to the Repository, accessed February 16, 2026, https://git-scm.com/book/ms/v2/Git-Basics-Recording-Changes-to-the-Repository
9. Mastering .gitignore: Keep Your Git Repo Clean and Professional - DEV Community, accessed February 16, 2026, https://dev.to/charan_gutti_cf60c6185074/mastering-gitignore-keep-your-git-repo-clean-and-professional-c3h
10. .gitignore Command Guide: Practical Examples and Terraform Tips | env zero, accessed February 16, 2026,

https://www.env0.com/blog/gitignore-command-guide-practical-examples-and-terraform-tips
11. Why does GIT ignore tmp folders? - Stack Overflow, accessed February 16, 2026, https://stackoverflow.com/questions/63521694/why-does-git-ignore-tmp-folders
12. How to Use "Sparse Checkout" to Manage Large Git Repositories, accessed February 16, 2026, https://www.git-tower.com/learn/git/faq/git-sparse-checkout
13. How to Configure Git Sparse Checkout - OneUptime, accessed February 16, 2026, https://oneuptime.com/blog/post/2026-01-24-git-sparse-checkout/view
14. Git - Difference Between 'assume-unchanged' and 'skip-worktree' - Stack Overflow, accessed February 16, 2026, https://stackoverflow.com/questions/13630849/git-difference-between-assume-unchanged-and-skip-worktree
15. Difference Between assume-unchanged and skip-worktree in Git | Baeldung on Ops, accessed February 16, 2026, https://www.baeldung.com/ops/git-assume-unchanged-skip-worktree
16. Telemetry - Visual Studio Code, accessed February 16, 2026, https://code.visualstudio.com/docs/configure/telemetry
17. .gitignore file - ignoring files in Git | Atlassian Git Tutorial, accessed February 16, 2026, https://www.atlassian.com/git/tutorials/saving-changes/gitignore